2 Introduction to Big O Time Complexity

What Is Complexity Analysis?

Complexity analysis measures **how efficient an algorithm is**.
It helps answer:

- How much **time** an algorithm takes
- How much **memory** an algorithm uses

Example problem

```
input = N
output = (N-1) + (N-2) + ... + 1

N = 5
Approach 1
4 + 3 + 2 + 1 (iterate from (n-1) to 1 and keep adding) => 10

Approach 2
Sum Formula
1 + 2 + 3 + ⋯ + (N−1) = (N−1)N/2
```

Which Approach Is Better?

Why care about identifying which is better?

Huge amount of data gives significant performance difference between solution.

What does better means?

- Fast => Time Complexity
- Less memory => Space Complexity

Complexity Analysis, Big O

Important Concept: Time Is NOT Measured in Seconds

We do **not** measure algorithm efficiency by stopwatch time.
Because:

- Different computers have different speeds
- Different languages run differently
- System load affects timing

How We Measure Time Complexity

We measure by:
**Number of simple operations performed
Examples of operations:

- Addition
- Comparison
- Assignment
- Array access

$$(N-1)*N/2$$

Number of operation performed = 3 OP so complexity will be O(3) → Constant Work
Loop Example if N = 5

$$4+3+2+1$$

n - 1 operation for each digits
n - 2 operation for each operator
so the total operations are

$$2n-3$$

If N = 5 so number of operations are 7.
If N= 1000 so number of operations are

$$2*1-3\ \ 1$$

If we use n*(n-1)/2 there are 3 OP only*
$$ 1000 (1000 - 1) / 2 $$
As input size increases, how does the number of operations grow?
This is the core idea of **Time Complexity**.

Time Complexity

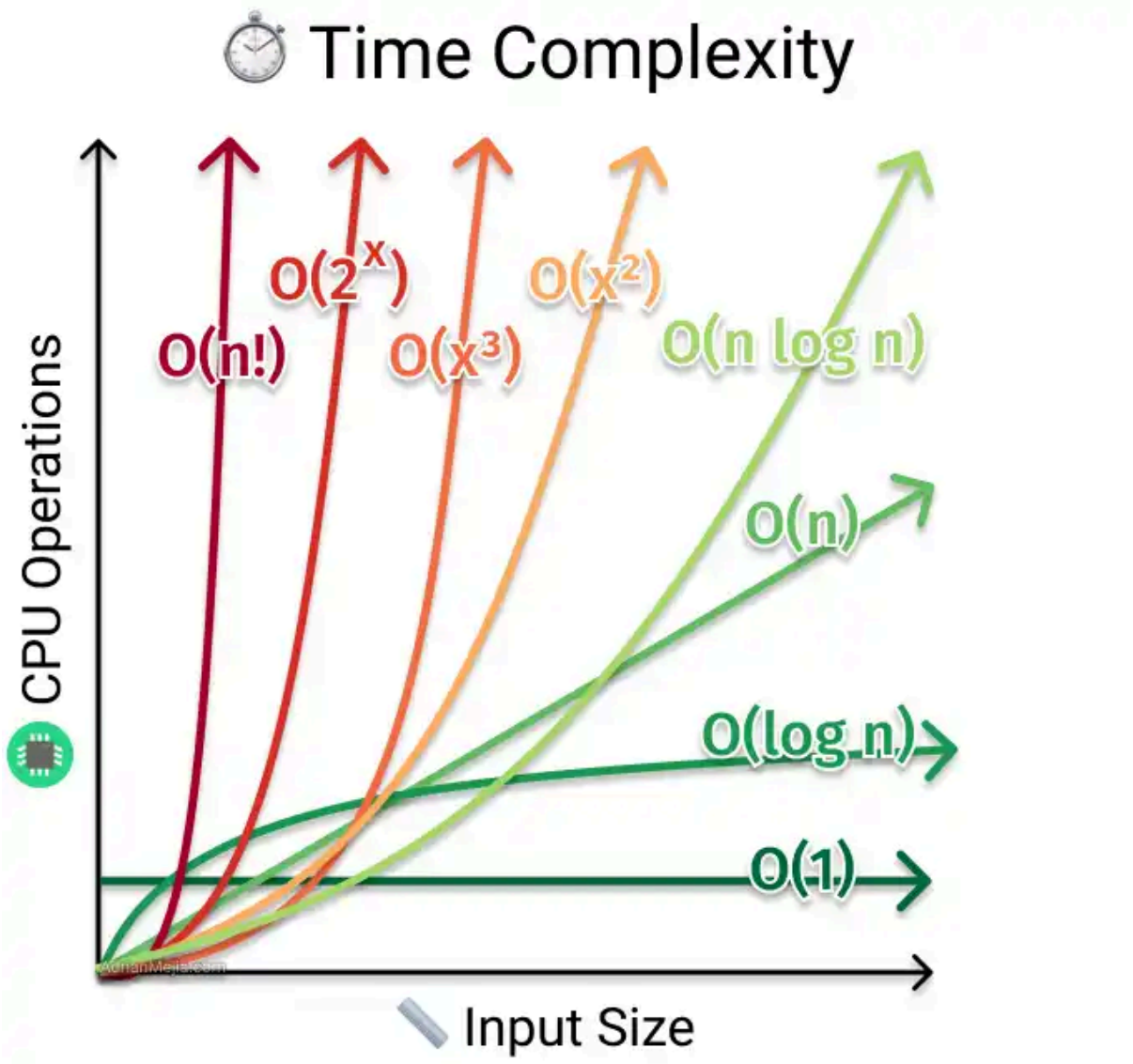
Time complexity measures how the runtime of an algorithm grows as input size (N) increases.
To identify time complexity we do asymptotic analysis.
And asymptotic expression is expressed using Big O notation.

Asymptotic Analysis

For I/P size N -> number of operation has to done by the computer is f(N).

$$()\ ^{2}+1$$

We can test the growth by give the value of x.
When we do complexity analysis we are not interested in the precision we are interested in the trend.



For identifying the trend we use asymptotic analysis.

$$(n) \;\; n + 3$$

- if n = 1 => no. of operation are 4
- if n = 1000 => no. of operation are 1003
- But suppose if N is very large and equation is (N + 3).
- "This 3 becomes insignificant so we can ignore it because we are not checking the precision we want a trend so it become".

  $$(n) \;\; n$$

  So we can say that the time complexity of the order on N or O(N).
  O(N) => This tells as I/P size increases, time take increases linearly.

So asymptotic analysis says that we are interested in the properties of a function f(N) as N become very large.

Example to solve

Techniques to simplify Big O Expression.

- Drop constant $(2N^2)$ (N^2)
- Drop insignificant terms $(N^3 + 1N)$ (N^3)
- Different input parameters no drop $(N^3 +)$

Logarithm

Algorithm that cuts I/P in half at every step then that relation is log n.

Example

$-$ $4-$ $2-$ 1 3 checks only. => 3
If you double I/P we have to perform only 1 extra operation.
$1-$ $-$ $4-$ $2-$ 1 => 4 checks only.
n => It means $_2 n$ in coding.
$_2 1$ => 2^4 1
$_2$ 2^3

How log is better than N?

2^1 124
2^2 14

- n increased 1
- log n increased only 10 -> 20

$$(n) \quad n^2 + 3n$$

$$(n) \quad 2n^2 + n$$

$$(n) \quad 2n^2 + n$$

$$(n) \quad 3n + 1$$

$$(n) \quad n^3 + 2n^2 + n$$

$$(n) \quad n + 2$$

$$(n) \quad n^2 + 1n +$$

$$(n) \quad 4$$

$$(n) \quad n \; n + n$$

$$(n) \quad 1n^2 + 3n^3$$

$$(n) \quad 2^n + n^2$$

$$(n) \quad n + n^3$$

$$(n) \quad n + n$$

$$(n) \quad n^2 \; n + n^2$$

$$(n) \quad 3 \; n + n$$

$$(n) \quad n^4 + n^3 + n$$

$$(n) \quad n + (n^2)$$

$$(n) \quad n + n \; n$$

$$(n) \quad 2^n + 3^n$$

$$(n) \quad n^1 + n$$

$$(n) \quad 1n +$$

$$(n) \quad n^2 + n \; n$$

Case	Complexity
loop inside loop	multiply
loops one after another	add

```
# 1
for i in range(n):
    print(i)

# 2
for i in range(n):
    for j in range(n):
        print(i, j)

# 3
i = 1
while i < n:
    i = i * 2

# 4
i = n
while i > 0:
    i = i // 2

# 5
for i in range(n):
    for j in range(i):
        print(i, j)

# 6
for i in range(n):
    for j in range(n):
        for k in range(n):
            print(i, j, k)

# 7
for i in range(n):
    print(i)
```

```python
for j in range(n):
    print(j)

# 8
for i in range(n):
    for j in range(1, n, 2):
        print(i, j)

# 9
for i in range(n):
    j = 1
    while j < n:
        j *= 2

# 10
for i in range(n):
    j = n
    while j > 0:
        j //= 2

# 11
for i in range(n):
    for j in range(i, n):
        print(i, j)

# 12
for i in range(n):
    for j in range(n):
        break

# 13
for i in range(n):
    if i % 2 == 0:
        print(i)

# 14
i = 1
while i < n:
    j = 1
    while j < n:
        j *= 2
    i *= 2

# 15
for i in range(n):
    for j in range(n):
        if j == n-1:
            print(i)

# 16
```

```python
for i in range(n):
    j = i
    while j > 0:
        j -= 1

# 17
for i in range(n):
    for j in range(n):
        if j % 2 == 0:
            print(j)

# 18
i = n
while i > 1:
    i = i ** 0.5

# 19
for i in range(n):
    for j in range(i*i):
        print(j)

# 20
for i in range(n):
    for j in range(n):
        for k in range(j):
            print(i, j, k)
```

#	**Time Complexity**
1	O(n)
2	$O(n^2)$
3	O(log n)
4	O(log n)
5	$O(n^2)$
6	$O(n^3)$
7	O(n)
8	$O(n^2)$
9	O(n log n)
10	O(n log n)
11	$O(n^2)$
12	O(n)
13	O(n)
14	$O((\log n)^2)$

#	Time Complexity
15	O(n^2)
16	O(n^2)
17	O(n^2)
18	O(log log n)
19	O(n^3)
20	O(n^3)

```
for i in range(n):
    for j in range(i*i):
        print(j)
```

j will be = $+1+4+\ +1$
$^2+1^2+2^2+3^2+4^2$
$^2+1^2+2^2+\ +(n-1)^2\ \ (n-1)n(2n-1)/$
(n^3)

```
i = n
while i > 1:
    i = i ** 0.5
```

We want: How many times does the loop run? (value of k)

Each iteration: i → √i (So the value keeps decreasing.)

Iteration (k)	Value of i	Exponent
0	n	1
1	√n	1/2
2	√(√n)	1/4
3	√(n^(1/4))	1/8
4	n^(1/16)	1/16

After **k iterations**: $n^{(1/2)}$
Because exponent is multiplied repeatedly, not added.

When does loop stop?

Condition: i > 1
Loop stops when: 1
Substitute: $n^{(1/2)}\ \ 1$

We know: $n \quad 1$
So we want: $(1/2)$ → very small (close to 0)
But Exponent never becomes exactly 0
So we use **approximation**, not exact equality.

What is “small enough”?

We use a known useful value:

$$n^{1/n} \quad nn$$

So when exponent ≈ $1/n$, value becomes small (≈2)
We say:

$$(1/2) \quad 1/n$$

It is Not exact equality. Just a **threshold comparison**.
$1/2 \quad 1/n$
Multiply both sides by 2
$1 \quad 2/n$
Multiply both sides by n.
$2 \quad n$
Take log both side.
$(2) \quad (n)$
Apply log rule.
$2 \quad (n)$
so we get.

$$(n)$$

Linear

i = i - 1 => O(n)

Divide

i = i / 2 => O(log n)

Root

i = √i => O(log logn)

Operation	Effect
subtract	slow
divide	faster
√	**very fast (shrinks exponent)**

Time Complexity vs Constraints

Input Size (n)	O(1)	O(log n)	O(n)	O(n log n)	$O(n^2)$	$O(n^3)$	$O(2^n)$, O(n!)
$n \le 10$	Yes	Yes	Yes	Yes	Yes	Yes	Yes
$n \le 20$	Yes	Yes	Yes	Yes	Yes	Limit	Yes (small n)
$n \le 100$	Yes	Yes	Yes	Yes	Yes	Limit	No
$n \le 1{,}000$	Yes	Yes	Yes	Yes	Limit	No	No
$n \le 10^4$	Yes	Yes	Yes	Yes	Limit	No	No
$n \le 10^5$	Yes	Yes	Yes	Yes	No	No	No
$n \le 10^6$	Yes	Yes	Yes	Limit	No	No	No
$n \ge 10^7$	Yes	Yes	Limit	No	No	No	No

- **Yes** → Safe (won't TLE)
- **Limit** → Risky (depends on language / constants)
- **No** → Will likely TLE